



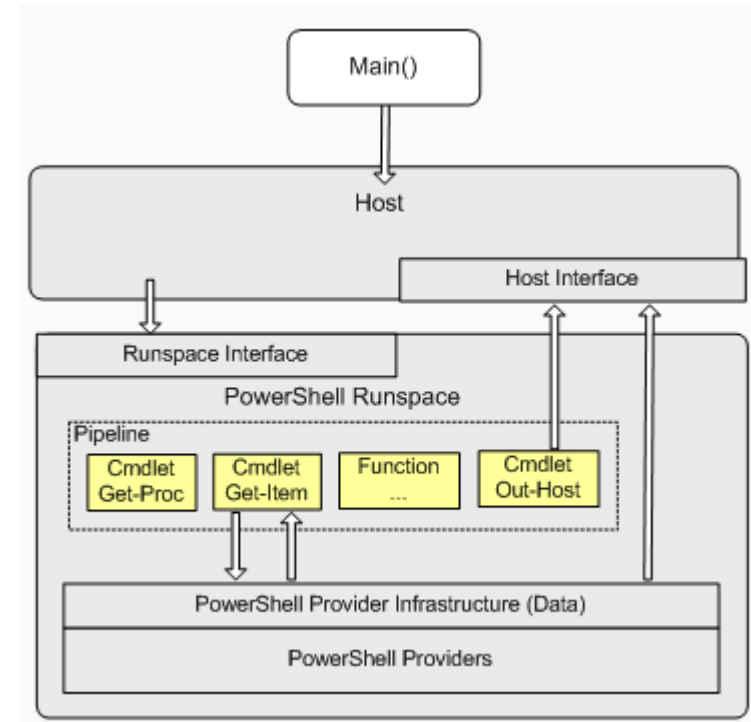
Runspaces

Module 9

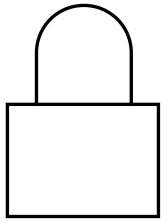
Runspace overview

What is a PowerShell Runspace

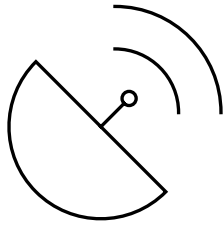
- An abstraction of the Windows PowerShell runtime used to simplify a user session
- Holds configuration information
 - Data about commands and PowerShell providers that the hosting application supports
 - Startup scripts for the runspace
- The Interface between the Host application and the user session



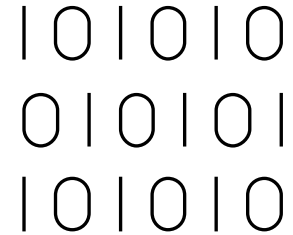
Runspace Use Cases



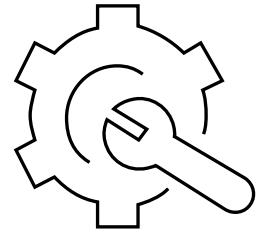
Running background task without locking the current runspace



Running background task but keeping the option to communicate with it

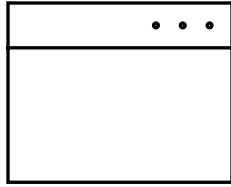


Remove the penalty of creating additional PowerShell processes with their own overhead



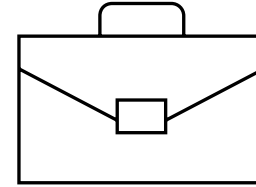
Ability to orchestrate resource consumption in parallel running runspaces

Runspace VS PowerShell Jobs



Runspace

- Light weight - Less resources consumed
- More control on initialization of environment
- More work to manage - Monitoring Completion, Disposing
- Additional thread in same process
- Call dispose method manually



Jobs

- Much overhead - Resource intense
- Less control over what is loaded on initialization
- Job Manager - Stop, Start, Suspend, Resume
- Start a new process host
- Cmdlet Remove-Job

Creating a Simple Runspace

A PowerShell Runspace class creates a new Runspace with all defaults

- Using the Type Accelerator **[PowerShell]**

```
$PowerShell = [PowerShell]::Create()  
$PowerShell.AddScript({Start-Sleep -Seconds 5; 'Done'})  
$PowerShell.Invoke()
```

- Checking the state after creation will show: Opened

```
$PowerShell.Runspace.RunspaceStateInfo
```

State	Reason
-----	-----
Opened	

Cleaning Up Runspace resources



A runspace needs to be cleaned up to free resources

Open file handles

Open Network connections

Open Database connections

Assigned variables



Cleaning a runspace works by calling the method **Dispose**

Cleaning Up Runspace resources - Example

- Starting the runspace

```
$Powershell = [PowerShell]::Create()  
$Powershell.AddScript({Start-Sleep -Seconds 5; 'Done'})  
$Powershell.Invoke()
```

- Disposing the runspace

```
$Powershell.Dispose()
```

- Removing the references to the variable

```
$Powershell.Dispose()
```


Demonstration

A Simple Runspace



Questions?



Runspace commands

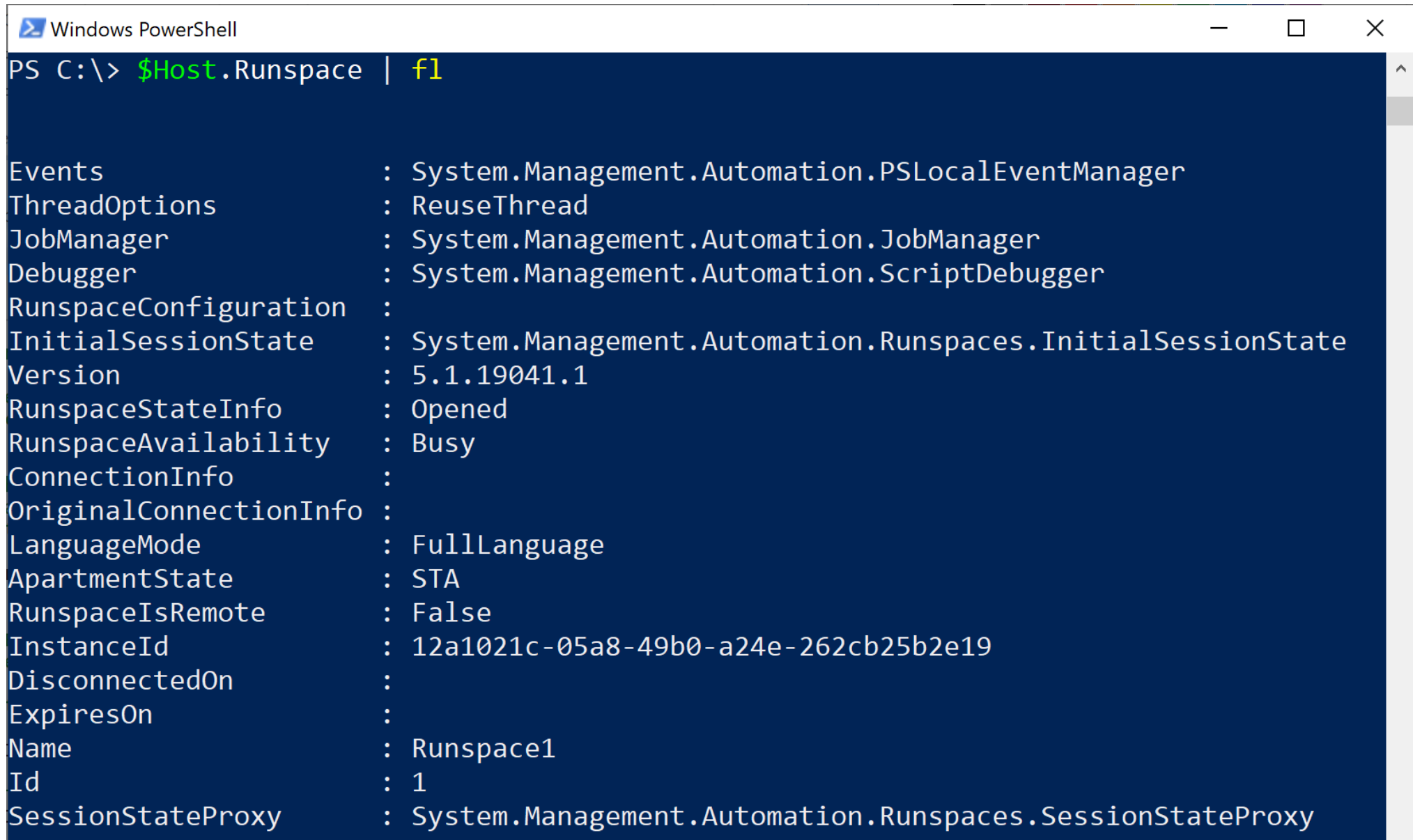
Runspace Commands

Name	Logic	Example
Get-Runspace	Shows runspaces in the current host	<pre>PS C:\> Get-Runspace Id Name ComputerName -- ---- - 1 Runspace1 localhost</pre>

There is only 1 cmdlet for runspaces. Most actions are handled by the methods of the classes used.

PowerShell.exe Default Runspace

Inspecting the PowerShell.exe Runspace



```
Windows PowerShell
PS C:\> $Host.Runspace | fl

Events                : System.Management.Automation.PSLocalEventManager
ThreadOptions         : ReuseThread
JobManager            : System.Management.Automation.JobManager
Debugger              : System.Management.Automation.ScriptDebugger
RunspaceConfiguration :
InitialSessionState   : System.Management.Automation.Runspaces.InitialSessionState
Version               : 5.1.19041.1
RunspaceStateInfo     : Opened
RunspaceAvailability  : Busy
ConnectionInfo        :
OriginalConnectionInfo :
LanguageMode          : FullLanguage
ApartmentState        : STA
RunspaceIsRemote      : False
InstanceId             : 12a1021c-05a8-49b0-a24e-262cb25b2e19
DisconnectedOn        :
ExpiresOn              :
Name                  : Runspace1
Id                    : 1
SessionStateProxy     : System.Management.Automation.Runspaces.SessionStateProxy
```

Inspecting the PowerShell.exe Runspace Session State

```
Windows PowerShell
PS C:\> $Host.Runspace.InitialSessionState

LanguageMode           : FullLanguage
TranscriptDirectory    :
ExecutionPolicy         : Restricted
UseFullLanguageModeInDebugger : False
ApartmentState         : Unknown
ThreadOptions          : D
ThrowOnRunspaceOpenError : F
AuthorizationManager    : M
```

Loaded Modules

Loaded Assemblies

Loaded PS Providers

```
Modules                : {PSReadline}
Assemblies              : {System.Management.Automation, Version=3.0.10.0, Culture=neutral,
                          PublicKeyToken=31bf3856ad364e35, ProcessArchitecture=MSIL,
                          Microsoft.PowerShell.Security}
Types                  : {C:\Windows\System32\WindowsPowerShell\v1.0\types.ps1xml,
                          C:\Windows\System32\WindowsPowerShell\v1.0\typesv3.ps1xml}
Formats                : {*, *, *, *...}
DisableFormatUpdates   : False
Providers              : {Registry, Alias, Environment, FileSystem...}
Commands               : {*, *, more, cd....}
Variables              : {$, ^, StackTrace, OutputEncoding...}
EnvironmentVariables   : {}
StartupScripts         : {}
```

Loaded Startup Scripts

Loaded Environment Variables

Loaded commands and variables

Demonstration

Runspace Session States

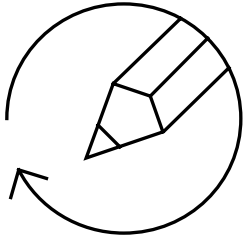


Questions?

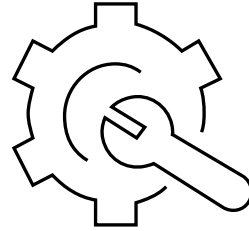


Creating and Initializing Runspaces

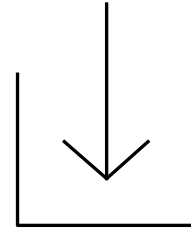
Creating and Initializing Custom Runspaces Flow



Initialize session
configuration classes
or states



Configure session
configuration classes
or states



Create a PowerShell
runspace using a
preconfigured session
configuration

Creating and Initializing Custom Runspaces – Example 2

Directly Configuring an Initialsessionstate



- Create a default initialsessionstate using its class

```
PS C:\> $InitialSessionState =  
[System.Management.Automation.Runspaces.InitialSessionState]::Create  
Default()
```

- Add a variable to the initialsessionstate object

```
PS C:\> $sessionVariable =  
[System.Management.Automation.Runspaces.SessionStateVariableEntry]::new  
("name", "value", $null)  
$InitialSessionState.Variables.Add($sessionVariable)
```

- Use the initialsessionstate upon creation of the runspace

```
PS C:\> $PowerShell = [powershell]::Create($InitialSessionState)
```

Creating and Initializing Custom Runspaces – Example 3

Using a PSSessionConfiguration file to initiate from



- Create a default initialsessionstate using its class

```
PS C:\> New-PSSessionConfigurationFile -Full -Path C:\psscconfig.pssc
```

- Edit the PSSC file as you see fit
(PSSC files are explained in detail in IT management workshop)
- Use the PSSC to create the initialsessionstate

```
PS C:\> $ISS =  
[System.Management.Automation.Runspaces.InitialSessionState]::Create  
FromSessionConfigurationFile("C:\psscconfig.pssc")
```

- Use the initialsessionstate upon creation of the runspace

```
PS C:\> $PowerShell = [powershell]::Create($ISS)
```

PowerShell Session Configuration Sample File

*psscconfig.pssc - Notepad

File Edit Format View Help

@{

Version number of the schema used for this document
SchemaVersion = '2.0.0.0'

ID used to uniquely identify this document
GUID = 'c2cc37a9-333e-4f9e-82a1-8f736e2c8476'

Author of this document
Author = 'mavangef'

Company associated with this document
CompanyName = 'Unknown'

Session type defaults to apply for this session configuration. Can be 'RestrictedRemoteServer' (recommended), 'Empty', or 'Default'
SessionType = 'Default'

Aliases to make visible when applied to a session
VisibleAliases = 'gci'

Cmdlets to make visible when applied to a session
VisibleCmdlets = 'Save-Help'

Functions to make visible when applied to a session
VisibleFunctions = 'Invoke-Function1', @{ Name = 'Invoke-Function2'; Parameters = @{ Name = 'Parameter1'; ValidateSet = 'Item1', 'Item2' },

External commands (scripts and applications) to make visible when applied to a session
VisibleExternalCommands = 'notepad.exe'

}

Demonstration

Runspace Session States



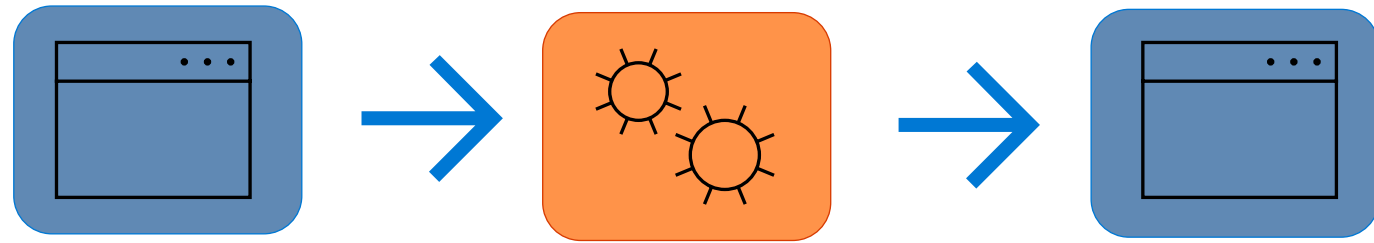
Questions?



Invoking Runspace Async and Sync

Synchronous Processing

Only 1 thread gets processed at a time



Runspace

GUI Shell

Synchronous Processing – Example



- Create a default runspace

```
PS C:\>$PowerShell = [PowerShell]::Create()
```

- Add a PowerShell script block

```
PS C:\>$PowerShell.AddScript({Start-Sleep -Seconds 5; 'Done'})
```

- Start the script block in **Synchronous** mode

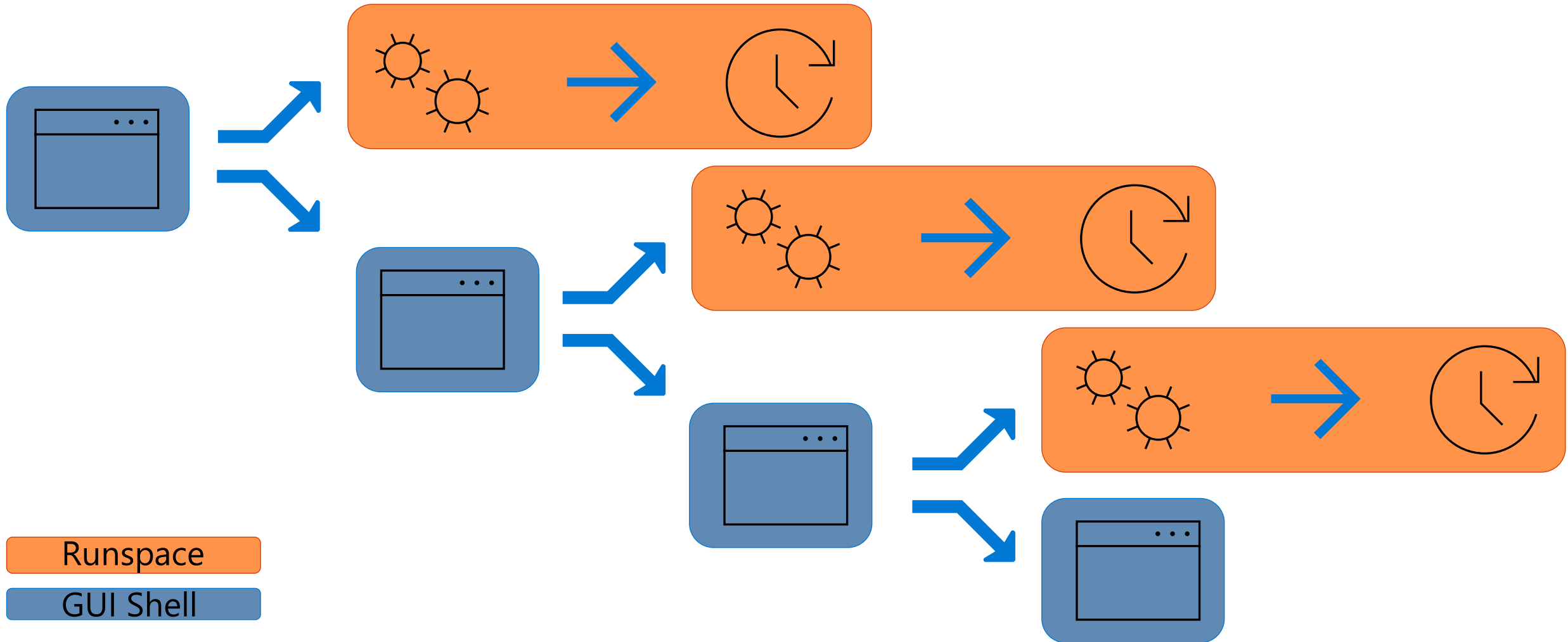
```
PS C:\>$PowerShell.Invoke()
```

- Shell is frozen during compute but displays results

```
Done
```

Asynchronous Processing

All runspaces process at the same time



ASynchronous Processing – Example

- Create a default runspace and Add Script block

```
PS C:\>$PowerShell = [PowerShell]::Create()  
PS C:\>$PowerShell.AddScript({Start-Sleep -Seconds 5; 'Done'})
```

- Start the script block in **ASynchronous** mode

```
PS C:\>$Async = $PowerShell.BeginInvoke()
```

- Wait till complete

```
PS C:\>$Async  
CompletedSynchronously IsCompleted AsyncState AsyncWaitHandle  
-----  
False True
```

- Retrieve the output from the runspace

```
PS C:\>$PowerShell.EndInvoke($Async)  
Done
```

Thread Safe Processing

- Making sure a Runspace does not interfere with another Runspace
- COM objects use ApartmentState: Multi Threaded Apartment / Single Threaded Apartment



Thread Safe



NOT Thread Safe

- Reading a log file
 - Write to SQL
- Writing into the same log file
 - Issue: File locks
 - Solution: 1 log per RunSpace

Demonstration

Runspace Session States



Questions?



Runspace pools

Types of Runspaces



Runspace

Single Thread

Synchronously and Asynchronously processing



Runspace Factory

Can use Runspace Pools

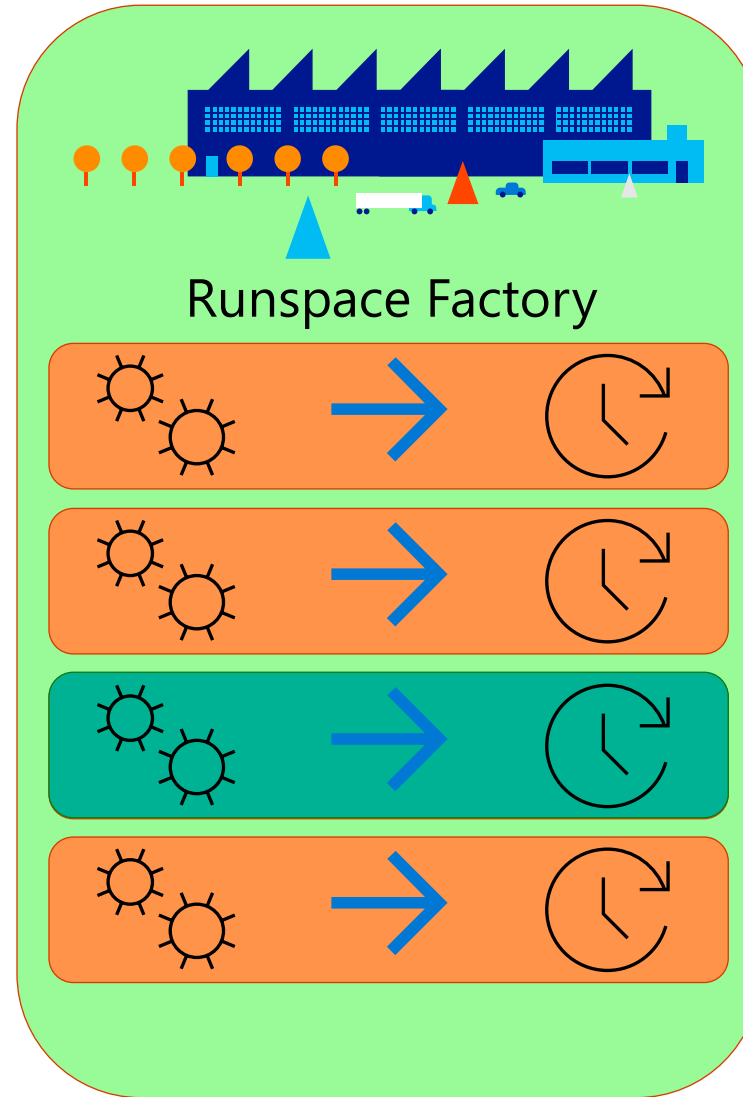
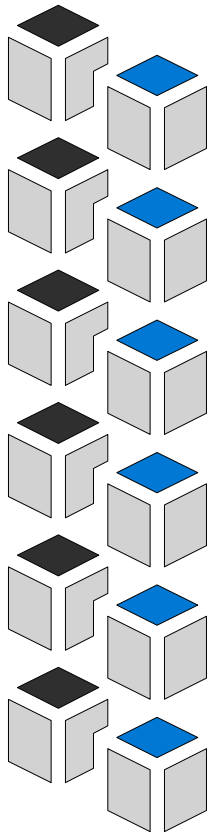
Control of Max nr of Pools

1 Thread per Runspace in the Pool

Synchronously and Asynchronously processing

Runspace Pools

Work Queue items
(PS Script blocks)



Creating and Initializing a Runspace Factory With RS Pools

- Creating a RunspaceFactory with Runspace Pools (Minimum 1 , Maximum 5)

```
PS C:\> $RunspacePool = [runspacefactory]::CreateRunspacePool(1,5)
```

- Open the Runspace when all configuration is done

```
PS C:\> $RunspacePool.Open()
```

- After opening the Runspace pool you can check availability

```
PS C:\> $RunspacePool.GetAvailableRunspaces()
```

- **Optional:** Create a list object to track runspaces

```
PS C:\> [System.Collections.ArrayList]$RunspaceList = @()
```

Assigning Script Blocks to the Runspace factory

```
foreach($RunspaceInstance in 1..20)
{
    #Create a runspace instance and assign it to the pool
    $PowerShellInstance = [powershell]::Create()
    $PowerShellInstance.RunspacePool = $RunspacePool

    #Add the script to the runspace
    [void]$PowerShellInstance.AddScript(
    {
        Get-ChildItem -Path C:\windows -Recurse
    })

    #Optional: Add all the data into a results table
    $RunspaceList.Add([pscustomobject]@{
        RunspaceInstance = $RunspaceInstance
        PowerShell       = $PowerShellInstance
        AsyncResult      = $PowerShellInstance.BeginInvoke()
    })
}
```

Viewing Runspace Progress, Stop Processing

- Using the optional list object to view progress

```
PS C:\> $RunspaceList
```

```
RunspaceInstance PowerShell
```

```
AsyncResult
```

```
-----
```

```
-----
```

```
1 System.Management...
```

```
System.Management....
```

```
2 System.Management...
```

```
System.Management....
```

```
PS C:\> $RunspaceList.AsyncResult
```

```
CompletedSynchronously IsCompleted AsyncState AsyncWaitHandle
```

```
-----
```

```
False
```

```
True
```

```
System.Threading...
```

```
False
```

```
False
```

```
System.Threading...
```

- Stopping / aborting all runspaces

```
PS C:\> $RunspaceList.PowerShell.Stop()
```

Demonstration

Runspace Pools



Questions?

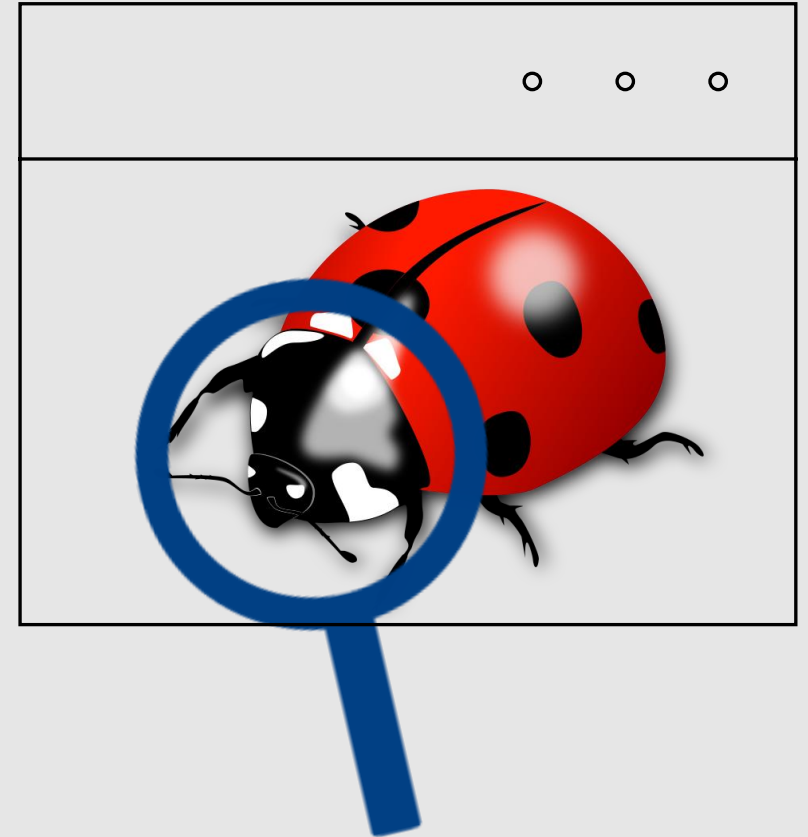


Debugging Runspaces

Debugging a Runspace

Local debugging:

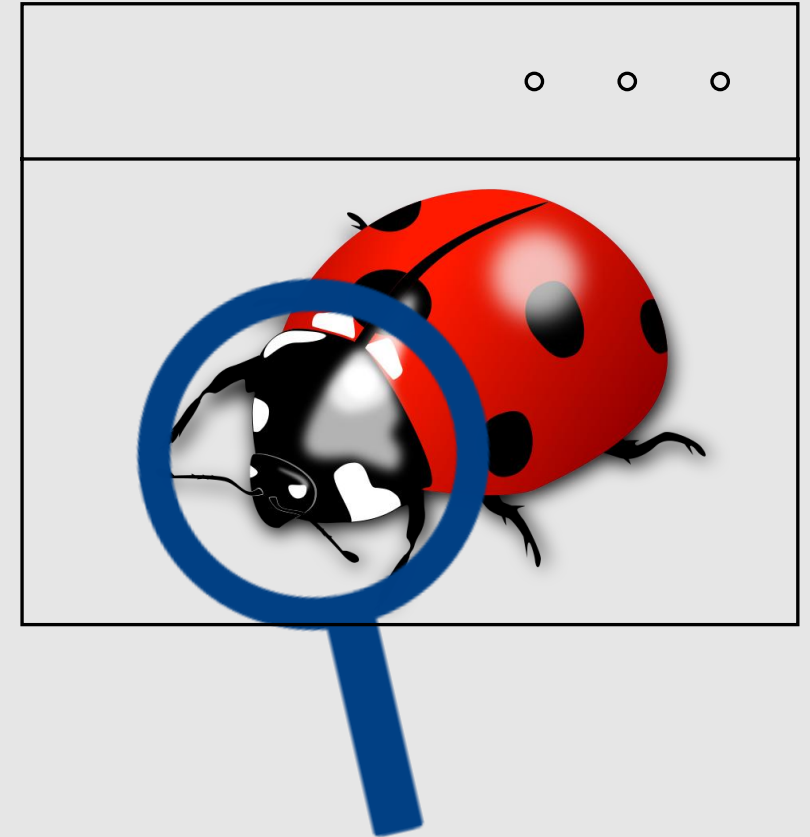
- Runspace uses the same debugger as scripts
- Available in ISE and Shell
- **Debug-Runspace** cmdlet starts debug session with local or remote runspace
- Debugger needs to be attached to the runspace
- Use **Get-Runspace** to list runspaces within the current PowerShell hosts



Debugging a Runspace continued

Remote debugging:

- Need administrator access by default for remoting
- Debug runspace scripts, in the same way remote session scripts are debugged



Debugging example

- Create a Runspace and name it (easy to attach debugger per name)

```
PS C:\> $ps = [powershell]::Create()  
PS C:\> $ps.Runspace.name = "debugtest"
```

- Add a script and invoke the runspace async

```
PS C:\> $ps.AddScript({do{$2 = 1 + $2 ;start-sleep -s 5}while($true)})  
PS C:\> $async = $ps.BeginInvoke()
```

- Attach the debugger to the runspace

```
PS C:\> Debug-Runspace -Name "debugtest"
```

- Debug

Debugging Runspace: debugtest

To end the debugging session type the 'Detach' command at the debugger prompt, or type 'Ctrl+C' otherwise.

```
Stopped at: do {$2 = 1 + $2 ; start-sleep -seconds 5}while($true )  
[DBG]: [Process:34644]: [debugtest]: PS C:\WINDOWS\system32>> L
```

```
1:* do {$2 = 1 + $2 ; start-sleep -seconds 5}while($true )
```

Runspace streams

- All streams are available in the runspace
- View them via the runspace object

```
PS C:\> $PowerShell = [PowerShell]::Create()
PS C:\> $powershell.AddScript({1/0})
PS C:\> $PowerShell.Invoke()
PS C:\> $PowerShell.Streams
```

```
Error      : {Attempted to divide by zero.}
Progress   : {}
Verbose    : {}
Debug      : {}
Warning    : {}
Information : {}
```

	1 output / Success
	2 Error
	3 Warning
	4 Verbose
	5 Debug
	6 Information

Demonstration

Runspace Debugging



Questions?



Runspaces

60 minutes



